

Assignment requirement checklist

Verified Docker Desktop run | 2026-09-23 | UTC and Pakistan Standard Time (UTC+05:00)

Fresh verification: 23 September 2026. Tests 26/26; concurrency 1/19. Current evidence is in VERIFICATION.pdf and docs/evidence/latest. Historical staged-load and outage checks remain explicitly dated in the earlier record. All 15 original screenshots are included in the report appendix.

Assignment requirement checklist

Status: **Verified Docker** means an actual container/API/Prometheus/Elasticsearch check ran on this machine; **Documented** means an explanation was reviewed. User-captured browser screenshots are included in the report appendix. See VERIFICATION.md and the linked raw evidence for exact scope and limits.

0 Inspect existing repository, preserve work, OS

Implementation: Empty repository inspected; Windows PowerShell detected, no prior app overwritten

File/location: Verification record

How to verify: Read environment notes

Status: Verified; prior work preserved

1 Room service and API-only UI

Implementation: Rooms, availability, create/fetch/list/cancel, conflict, health/ready/metrics, Swagger

File/location: app/api/routes.py

How to verify: Smoke + /docs

Status: Verified Docker API/smoke; Swagger visual capture manual

2 Mature local stack

Implementation: Python 3.12/FastAPI/SQLAlchemy async/Postgres + complete requested observability stack/k6

File/location: Dockerfile, requirements, Compose

How to verify: Full startup

Status: Verified Docker, all required services healthy

3 HTTP/validation/exception handling/UTC

Implementation: Typed schemas, clean errors, correlation, lifecycle/timeouts

File/location: app/schemas, main.py, middleware

How to verify: API tests

Status: Verified Docker/PostgreSQL tests

3 Pool, transactions, indexes, schema

Implementation: Bounded engine pool, short transactions, deterministic versioned initialization

File/location: app/db, repositories

How to verify: Integration tests

Status: Verified Docker/PostgreSQL tests

3 No double-booking race

Implementation: Partial GiST exclusion constraint

File/location: app/db/schema.sql

How to verify: 20-way test + direct SQL test

Status: Verified Docker/PostgreSQL tests

4 Performance measurability/bounded labels

Implementation: Throughput/status/latency/concurrency/DB/business/resource data

File/location: Metrics, three dashboards

How to verify: k6 baseline, Prometheus

Status: Verified Docker baseline and staged load; capacity limit recorded

5 Counter/Gauge/Histogram/Summary

Implementation: Required metric types plus business and logging health metrics

File/location: app/observability/metrics.py

How to verify: /metrics, tests, report table

Status: Verified metric types and real samples

5 Business metrics/own exploration

Implementation: Created/cancelled/conflicts/active/lead time; derived conflict percentage

File/location: Service, refresh loop, app dashboard

How to verify: Flow + conflict traffic

Status: Verified metric types and real samples

6 All sixteen application dashboard items

Implementation: 20 panels with titles, descriptions, units, legends and queries

File/location: observability/grafana/dashboards/campus-app.json

How to verify: Open provisioned dashboard

Status: 20 application panels/queries verified; browser capture manual

7 Throughput/load dashboard

Implementation: k6 VUs and client tails, server tails/rates, in-flight, resources, DB

File/location: campus-load.json

How to verify: Remote-write baseline

Status: 14 load panels/queries verified with real remote write; screenshot included

8 Node Exporter + named measured machine

Implementation: Linux VM collector mounts and explicit scope limitations

File/location: Compose, Prometheus, campus-node.json, README

How to verify: Targets / resource panels

Status: Verified Docker Linux/WSL2 scope and six resource panels

9 Prometheus config/scrapes/storage

Implementation: 5s scrape, application/node/self/optional demo, named volume

File/location: observability/prometheus/prometheus.yml

How to verify: promtool check config; targets

Status: Verified config and live targets

10 Structured JSON logs/correlation/privacy

Implementation: Allowlisted JSON, safe IDs, error IDs, bounded queue, no bodies/secrets

File/location: Logging/middleware/main

How to verify: Unit tests; inspect real file

Status: Verified tests and indexed correlated JSON

11 Filebeat→Elasticsearch→Kibana

Implementation: Shared volume, NDJSON, explicit typed mapping, data view setup/KQL

File/location: Filebeat/Elastic/Kibana config, scripts

How to verify: Filebeat test config; verify-stack.py, Discover

Status: Verified Filebeat→Elasticsearch; Kibana view/mappings verified, screenshot included

12 Persistence/retention/ILM

Implementation: Size rotation, named volumes, daily rollover + seven-day deletion policy

File/location: ILM JSON, README/report

How to verify: _ilm/explain; long-duration expiry observation

Status: Volumes and ILM attachment verified; long-term expiry unobserved

13 Metric end-to-end

Implementation: Exact observation function, value, exposition, scrape/storage/query/panel

File/location: Report Part D

How to verify: Follow one actual sample

Status: Verified actual sample/query; screenshot included

14 Log end-to-end

Implementation: ID→JSONL→parsed fields→index→KQL; verifier emits real stored hit

File/location: Report Part C/D, verifier

How to verify: Run smoke/verifier + Discover

Status: Verified actual stored hit and request ID; screenshot included

15 Architecture/protocols/storage/failure effects

Implementation: Mermaid diagrams and eight-component failure matrix

File/location: README, report Part D

How to verify: Read; controlled stop/start

Status: Documented; outage/recovery verified 21 September

16 Five reproducible business k6 tests

Implementation: Smoke/baseline/load/stress/concurrency; correct expected statuses

File/location: load-tests/

How to verify: k6 inspect, execute profiles

Status: Five profiles parsed; Docker baseline/load/concurrency and PowerShell smoke executed

17 Increasing users experiment

Implementation: 10/25/50/100/200 VUs with plateaus; comparison instructions

File/location: load.js, report E1

How to verify: Load dashboard/time ranges

Status: Verified 21 September; not rerun on 23 September

18 Safe anomaly/recovery automation

Implementation: Local-only delay every fifth business request; PS/Bash cleanup/time ranges

File/location: Middleware, anomaly scripts

How to verify: Run baseline/fault/recovery

Status: Verified Docker baseline/fault/recovery

19 Isolated cardinality experiment

Implementation: Separate app, hard cap100, restart without label, real query waits

File/location: experiments/cardinality.py, scripts

How to verify: Run both modes; count100→1

Status: Verified actual Docker/Prometheus 100→1

20 Real concurrency proof

Implementation: Exactly one winner and nineteen409; DB count; k6 batch

File/location: tests/test_integration.py, concurrency.js

How to verify: PostgreSQL suite + k6

Status: Verified Docker/PostgreSQL and final k6 1 winner/19 conflicts

21 Required automated tests

Implementation: Health/ready/booking lifecycle/validation/conflict/IDs/metrics/errors/timeouts/fault/concurrency

File/location: tests/

How to verify: pytest or isolated Compose tests

Status: 26 Docker tests passed, no skips

22 Compose complete services/healthchecks

Implementation: All eight requested services + deterministic setup/optional tooling

File/location: docker-compose.yml

How to verify: docker compose config; build/up

Status: Verified build/start/health

23 Local resource stability

Implementation: Memory limits, ES512MB heap/single node, bounded queues/pool/logs

File/location: Compose, configs

How to verify: docker stats --no-stream, load

Status: Measured VM/app resources; no OOM in final containers

24 Observability failure independence

Implementation: App startup depends only on PostgreSQL; local queued logs and pull metrics

File/location: App + Compose; report matrix

How to verify: Stop Grafana/Filebeat, smoke

Status: Verified 21 September; not repeated during this refresh

25 Complete Windows-friendly README

Implementation: Exact PowerShell/Bash startup/API/tests/experiments/cleanup/limits

File/location: README.md

How to verify: Fresh-clone walkthrough

Status: Docker walkthrough executed; browser visuals manual

26 Report Parts A-E

Implementation: Detailed metrics table, logs, design, two experiments, honest placeholders

File/location: report.md

How to verify: Read alongside evidence

Status: Updated with real Docker observations; all 15 screenshots included

27 Rubric-weighted effort

Implementation: API kept small; focus metrics/logs/design/experiments

File/location: Repository

How to verify: Compare A10/B30/C25/D20/E15

Status: Implemented scope

28 Screenshot checklist

Implementation: Actions and evidence for every required chart/search/experiment

File/location: docs/DEMO-CHECKLIST.md

How to verify: Follow checklist

Status: Exact historical browser ranges/panels/KQL supplied; screenshots included

29 Viva questions

Implementation: All listed concepts in plain technical language

File/location: docs/VIVA-NOTES.md

How to verify: Rehearse with code/charts

Status: Documented

30 Troubleshooting

Implementation: Ports/memory/ES/Kibana/Filebeat/Grafana/targets/DB/volumes/no traffic

File/location: README

How to verify: Run diagnosis commands as needed

Status: Documented; actual integration fixes recorded

31 Safe cleanup/gitignore

Implementation: down preserves data; -v deletion explained; runtime files ignored

File/location: README, .gitignore, .dockerignore

How to verify: Inspect instructions

Status: Ignore rules verified; named runtime volumes excluded

32 Security/privacy

Implementation: Loopback, no personal data/secrets logged, .env excluded, local limitations explicit

File/location: README, .env.example, logging

How to verify: Tests/review; no internet exposure

Status: Local scope reviewed; sensitive runtime files excluded from Git

33 Lint/tests/config/build/start checks

Implementation: Ruff, pytest, Compose and promtool; dedicated Docker suite

File/location: Verification record

How to verify: Exact recorded commands

Status: Ruff/config/build/start and 26 container tests passed

34 Actual modest performance validation

Implementation: Native/Compose k6 runner with JSON evidence; no guessed speedup

File/location: results, verification record

How to verify: Actual baseline summary

Status: Real Docker baseline/resource results saved

35 No fake evidence

Implementation: Measured values scoped, placeholders for unexecuted full-stack runs

File/location: Report + verification

How to verify: Trace every number to a saved run

Status: Actual evidence preserved; no fabricated screenshots

36 Clean organization

Implementation: API/core/DB/repositories/schemas/services/observability/tests/docs/scripts

File/location: Repository tree

How to verify: Inspect files

Status: Implemented

37 Straightforward style

Implementation: Small functions, no added services without a role, lint/format

File/location: App + scripts

How to verify: Ruff and code review

Status: Lint/format passed

38 Metric/log correctness details A-I

Implementation: Template/method/status bounds, monotonic clocks, finally cleanup, correlation, scrape/probe exclusion

File/location: Middleware + tests

How to verify: Metric-label/error tests

Status: Verified tests, exposition and live pipeline

39 Database performance

Implementation: Indexed range exclusion and queries, bounded pagination/pooling, no N+1

File/location: SQL/repository/engine

How to verify: Integration/load + query inspection

Status: DB invariants verified; pool saturation measured, tuning unclaimed

40 Valid load data

Implementation: Unique iteration slots, cleanup, controlled409; separate unexpected Rate

File/location: common.js, concurrency script

How to verify: Inspect and run k6

Status: Verified real workload; unexpected responses separate from deliberate conflicts

41 HTTP error classification

Implementation: 2xx/409/422/5xx separate; explicit server failure rate

File/location: App/dashboard/report

How to verify: Conflict + validation + DB failure

Status: Verified 409s, validation tests and controlled 503s

42 Startup experience

Implementation: Copy env, build/up, auto DB/Grafana/ES; one deterministic Kibana setup

File/location: README, Compose

How to verify: Fresh Docker host walkthrough

Status: Complete startup/Kibana setup verified

43 Final self-audit

Implementation: This complete section-numbered table

File/location: This file

How to verify: Compare against pasted request

Status: Updated after real Docker execution

44 Completion report

Implementation: Commands, files, URLs, remaining checks in final response

File/location: Final response + README

How to verify: Read delivery

Status: Results, limits and manual screenshots documented

45 Autonomous completion and honest errors

Implementation: Implementation/config/tests/docs/experiments carried through; limitations disclosed

File/location: Verification record

How to verify: Inspect results/pending tasks

Status: Docker work completed; user-captured browser evidence included

All requested Docker runtime experiments are now recorded with real evidence. Original screenshot evidence is included; source, PDFs and screenshots are ready for submission. Seven-day deletion and production capacity are not claimed.